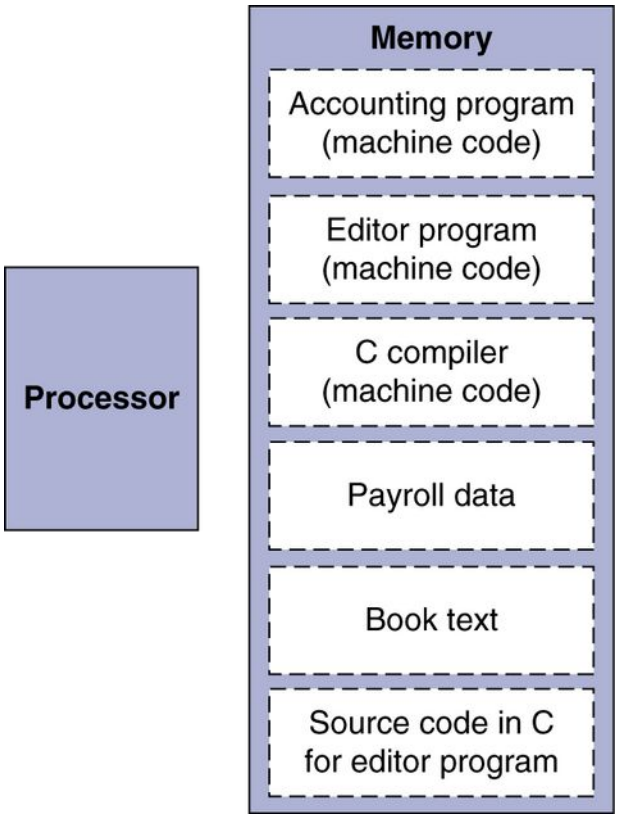
Chapter 2

Instructions: Language of the Computer

Stored Program Computers

The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Stored Program Computers

The BIG Picture

- Instructions represented in binary, just like data
- Instructions and data stored



Features	Instruction	Data
Representation	Binary	Binary
Size	Fixed 32 bits	Depends on data type
Meaning	What CPU should do	What program processes
Interpretation	Defined by ISA (R / I / J format)	Defined by data type
Storage	Mainly Memory	Memory + Register File
CPU Access	Fetch → Decode → Execute	Load → Process → Store

- Standardized ISAs

Q1. A MIPS computer stores the following two 32-bit binary patterns in memory:
Pattern A is fetched and executed by the CPU.
Pattern B is loaded into a register and used in a calculation.
(a) What is the likely role of Pattern A and Pattern B?
(b) Although both are binary patterns stored in memory, why does the CPU treat them differently?

A1.

(a)Pattern A is an instruction, while Pattern B is data.

(b)Their roles and interpretations are different. An instruction is fetched, decoded, and executed, while data is used as an operand or result during computation.

Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

- Useful for extracting and inserting groups of bits in a word

Logical Operations

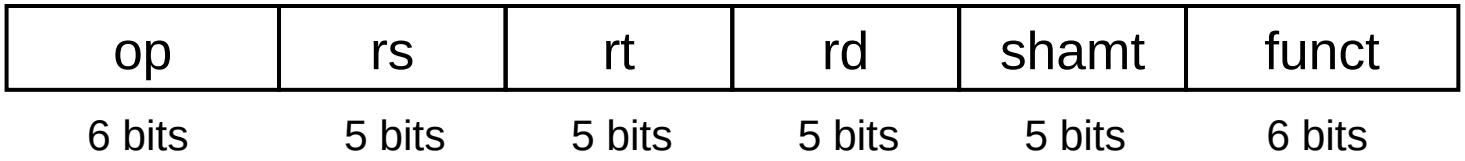
- Instructions for bitwise manipulation

Operation	C	Java	MIPS

	Operation (Chapter 2.3)	Operand (Chapter 2.1)
Meaning	Specifies what the CPU does	Specifies what data is used
Key Question	What to do?	What data / where?
Examples	add, sub, lw, sw	Register, Memory
Example	add in add $t0, $t1, $t2	$t1, $t2 are register operands
This Section	and, or, nor, sll, srl	Logical operations still work on operands

- Useful for extracting and inserting groups of bits in a word

Shift Operations



- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `sll` by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srl` by *i* bits divides by 2^i (unsigned only)

Shift Operations

Shift	Fill	Signed / Unsigned
Logical Left Shift	Right side → 0	Both, but watch overflow
Logical Right Shift	Left side → 0	Mainly unsigned
Arithmetic Right Shift	Left side → sign bit	Signed

- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `sll` by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srl` by *i* bits divides by 2^i (unsigned only)

Shift Operations

Shift	Fill	Signed / Unsigned
Logical Left Shift	Right side → 0	Both, but watch overflow
Logical Right Shift	Left side → 0	Mainly unsigned
Arithmetic Right Shift	Left side → sign bit	Signed

Q2. Assume all values are 8-bit signed two's complement numbers. Perform each shift and give the resulting binary and decimal values.

(a) Left shift 0010 1101 by 1 bit.

(b) Arithmetic right shift 0101 1000 by 2 bits.

(c) Arithmetic right shift 1011 0000 by 2 bits.

A2.

(a)0101 1010

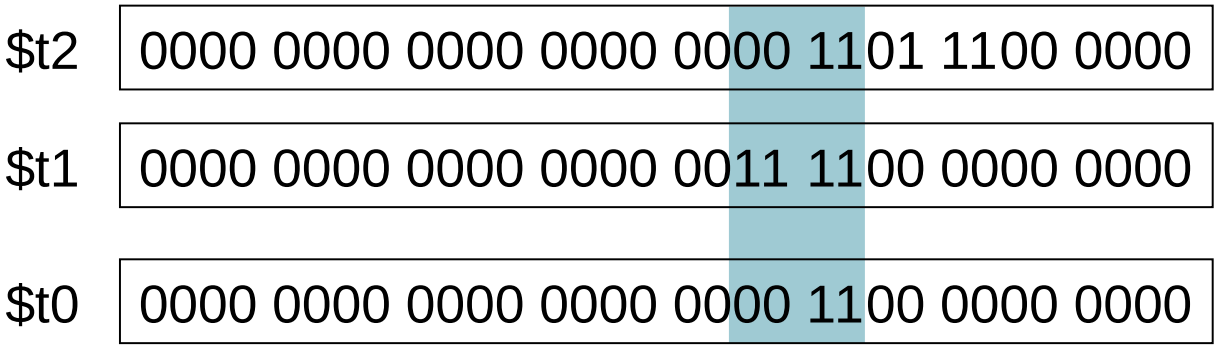
(b)0001 0110

(c)1110 1100

AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

```
and $t0, $t1, $t2
```



AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

Q3. Given the following 8-bit value 1101 1011
(a) Design an 8-bit mask to keep only bits 2–5 (bit 0 is the rightmost bit) and clear all other bits to 0.
(b) Perform the AND operation and give the result.

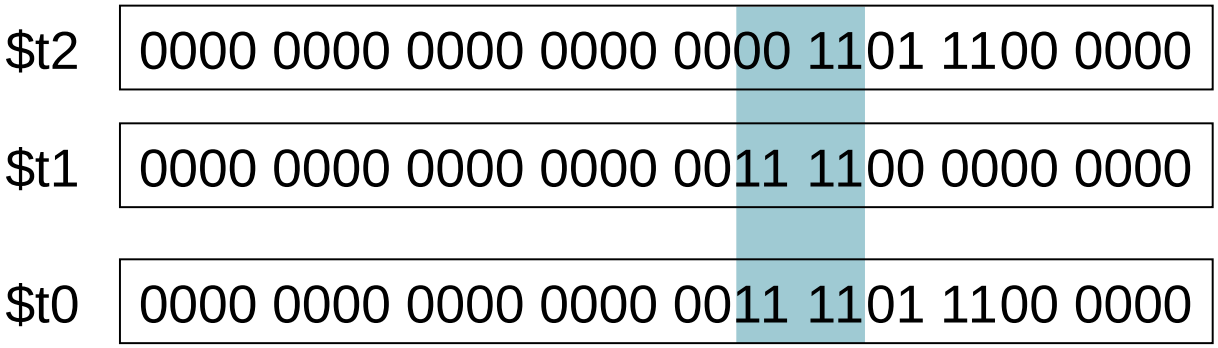
A3.

Bit:	**7 6 5 4 3 2 1 0**
Data:	**1 1 0 1 1 0 1 1**
Mask:	**0 0 1 1 1 1 0 0**
Result:	**0 0 0 1 1 0 0 0**

OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

```
or $t0, $t1, $t2
```



NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0
- MIPS has NOR 3-operand instruction
  - a NOR b == NOT (a OR b)

```
nor $t0, $t1, $zero
```

Register 0: always read as zero

$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	1111 1111 1111 1111 1100 0011 1111 1111

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs, rt, L1`
  - if (rs == rt) branch to instruction labeled L1;
- `bne rs, rt, L1`
  - if (rs != rt) branch to instruction labeled L1;
- `j L1`
  - unconditional jump to instruction labeled L1

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially

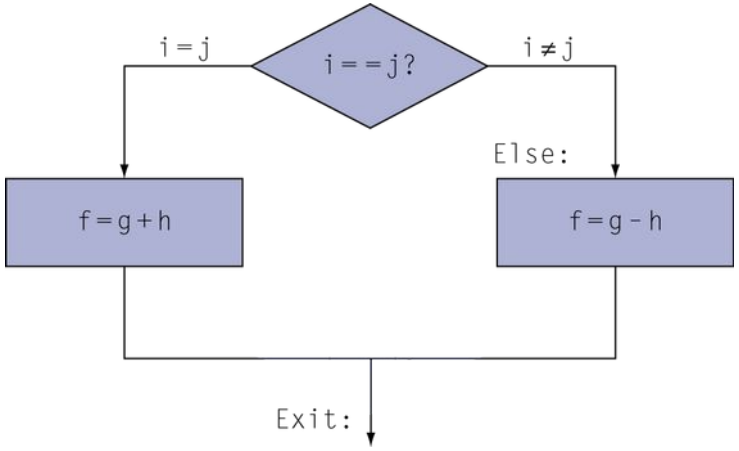
Category	Operations / Instructions	Purpose
Arithmetic	add, sub, addi	Perform arithmetic calculations
Data Transfer	lw, sw	Move data between memory and registers
Logical	sll, srl, and, or, nor	Manipulate individual bits
Conditional Branch	beq, bne	Change execution based on a condition
Unconditional Jump	j	Jump to another instruction

Compiling If Statements

- C code:

```
if (i==j) f = g+h;
else f = g-h;
```

  - f, g, … in $s0, $s1, …



- Compiled MIPS code:

```
      bne $s3, $s4, Else
      add $s0, $s1, $s2
      j   Exit
Else: sub $s0, $s1, $s2
Exit: …
```

Assembler calculates addresses

Compiling If Statements

Q4. Assume: a → $s0, b → $s1, x → $s2, y → $s3, z → $s4. Translate the following C code into MIPS assembly: if (a != b) x = y + z;
else x = y - z;

A4.

```
beq  $s0, $s1, Else
add  $s2, $s3, $s4
j    Exit
Else: sub  $s2, $s3, $s4
Exit:
```

```
Exit: …
```

Assembler calculates addresses

Compiling Loop Statements

- C code:

```
while (save[i] == k) i += 1;
```

  - i in $s3, k in $s5, address of save in $s6

- Compiled MIPS code:

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

Compiling Loop Statements

Q5. Assume: i → $s0, x → $s1, address of A → $s2
Each element of A is a 32-bit word. Translate the following C code into MIPS assembly:
while (A[i] == x) i = i + 1;

A5.

```
Loop: sll  $t0, $s0, 2
      add  $t0, $t0, $s2
       lw  $t1, 0($t0)
      bne  $t1, $s1, Exit
      addi $s0, $s0, 1
        j  Loop
Exit:
```

More Conditional Operations

- Set result to 1 if a condition is true
  - Otherwise, set to 0
- `slt rd, rs, rt`
  - if (rs < rt) rd = 1; else rd = 0;
- `slti rt, rs, constant`
  - if (rs < constant) rt = 1; else rt = 0;
- Use in combination with `beq`, `bne`

```
slt $t0, $s1, $s2  # if ($s1 < $s2)
bne $t0, $zero, L  #   branch to L
```

More Conditional Operations

Q6. Assume: $s1 = 12 $s2 = 20 $s3 = 5 Consider the following MIPS instructions:

```
slt  $t0, $s1, $s2
bne  $t0, $zero, L1
add  $s3, $s3, $s1
L1:  sub  $s3, $s3, $s1
```

(a) What value is stored in $t0 after slt?

(b) Is the branch to L1 taken?

(c) What is the final value of $s3?

A6.

(a) $t0=1

(b) Branch is taken

(c) $s3=–7

Branch Instruction Design

- Why not `blt`, `bge`, etc?
- Hardware for <, ≥, … slower than =, ≠
  - Combining with branch involves more work per instruction, requiring a slower clock
  - All instructions penalized!
- `beq` and `bne` are the common case
- This is a good design compromise

Branch Instruction Design

- Why not `blt`, `bge`, etc?
- Hardware for <, ≥, … slower than =, ≠
  - Combining with branch involves more work per instruction, requiring a slower clock

Q7. MIPS does not provide a basic blt instruction. Rewrite the following pseudo-instruction using slt and a branch instruction:

blt $s1, $s2, L1

A7.

```
slt  $t0, $s1, $s2
bne  $t0, $zero, L1
```

Signed vs. Unsigned

- Signed comparison: `slt`, `slti`
- Unsigned comparison: `sltu`, `sltiu`
- Example
  - $s0 = 1111 1111 1111 1111 1111 1111 1111 1111
  - $s1 = 0000 0000 0000 0000 0000 0000 0000 0001
  - `slt  $t0, $s0, $s1  # signed`
    - –1 < +1 ⇒ $t0 = 1
  - `sltu $t0, $s0, $s1  # unsigned`
    - +4,294,967,295 > +1 ⇒ $t0 = 0

Signed vs. Unsigned

- Signed comparison: `slt`, `slti`

Q8. Assume the following 8-bit values are stored in two registers: $s1 = 1111 1010 $s2 = 0000 0101 Determine the value of $t0 after each instruction: (a)slt $t0, $s1, $s2 (b) sltu $t0, $s1, $s2

A8.

For signed comparison:

$s1 = 1111 1010 = -6 $s2 = 0000 0101 = 5

$t0 = 1

For unsigned comparison:

$s1 = 1111 1010 = 250 $s2 = 0000 0101 = 5

$t0 = 0

Concluding Remarks

- Measure MIPS instruction executions in benchmark programs
  - Consider making the common case fast
  - Consider compromises

Instruction class	MIPS examples	SPEC2006 Int	SPEC2006 FP
Arithmetic	`add, sub, addi`	16%	48%
Data transfer	`lw, sw, lb, lbu, lh, lhu, sb, lui`	35%	36%
Logical	`and, or, nor, andi, ori, sll, srl`	12%	4%
Cond. Branch	`beq, bne, slt, slti, sltiu`	34%	8%
Jump	`j, jr, jal`	2%	0%

Section 2-3 Quick Questions

Q1. A MIPS computer stores two 32-bit patterns in memory. Pattern A is fetched and executed; Pattern B is loaded into a register for calculation. Identify their likely roles and explain why the CPU treats them differently.

Q2. For 8-bit signed two's-complement values, perform: (a) 0010 1101 << 1, (b) arithmetic right shift 0101 1000 by 2, and (c) arithmetic right shift 1011 0000 by 2.

Q3. For 1101 1011, design an 8-bit mask to keep only bits 2–5 (bit 0 is rightmost), then perform the AND operation.

Q4. Given a → $s0, b → $s1, x → $s2, y → $s3, z → $s4, translate into MIPS: if (a != b) x = y + z; else x = y - z;

Section 2-3 Quick Questions

Q5. Given i→$s0, x→$s1, and the address of A→$s2, translate into MIPS: while (A[i] == x) i = i + 1; Assume each A element is a 32-bit word.

Q6. Given $s1=12, $s2=20, $s3=5, trace: slt $t0,$s1,$s2; bne $t0,$zero,L1; add $s3,$s3,$s1; L1: sub $s3,$s3,$s1. Determine $t0, whether the branch is taken, and final $s3.

Q7. Rewrite the pseudo-instruction blt $s1, $s2, L1 using slt and a branch instruction.

Q8. Given $s1=1111 1010 and $s2=0000 0101, determine $t0 after (a) slt $t0,$s1,$s2 and (b) sltu $t0,$s1,$s2.

Exercise 1 | Bitwise Masking & Modification

Given the 8-bit value: Data = 1011 0110_2

(a) Design an AND mask to clear bits 4–6 (bit 0 is the rightmost bit) while keeping all other bits unchanged. Give the result.

(b) Starting from the result in (a), design an OR mask to set bits 0 and 3 to 1. Give the final 8-bit result.

(c) Assume Data is in $t1, the AND mask is in $t2, and the OR mask is in $t3. Write the two MIPS instructions that store the final result in $t0.

Answer

(a) AND mask = $1000\ 1111_2$
$1011\ 0110_2$ AND $1000\ 1111_2$ = $1000\ 0110_2$

(b) OR mask = $0000\ 1001_2$
$1000\ 0110_2$ OR $0000\ 1001_2$ = $1000\ 1111_2$

(c) and $t0, $t1, $t2
or $t0, $t0, $t3

Key Point: AND masks clear selected bits with 0s, while OR masks set selected bits with 1s.

Exercise 2 | Signed Shift Operations

Given two 8-bit signed two's-complement values:

A = 1101 1000_2 B = 0101 0000_2

(a) Interpret A and B as signed decimal values. Then perform an arithmetic right shift of A by 2 bits. Give the binary and decimal results.

(b) Perform a logical left shift of B by 1 bit. Give the resulting 8-bit binary value and interpret it as a signed decimal value.

Answer

(a) A = −40, B = 80

Arithmetic right shift: 1101 1000_2 >> 2 = 1111 0110_2 = −10

(b) Logical left shift: 0101 0000_2 << 1 = 1010 0000_2

As an 8-bit signed value, 1010 0000_2 = −96.

Key Point: Arithmetic right shift preserves the sign bit; left shift represents ×2 only when no overflow occurs.

Exercise 3 | Conditional Execution Trace

```
slt  $t0, $s0, $s1
beq  $t0, $zero, L1
add  $s2, $s2, $s0
j    Exit
L1:  sub  $s2, $s2, $s1
Exit:
```

Assume: $s0 = 15, $s1 = 9, $s2 = 4

(a) What value is stored in $t0? Is the beq branch taken?

(b) Which arithmetic instruction, add or sub, is executed?

(c) What is the final value of $s2?

Answer

(a) 15 < 9 is false, so $t0 = 0. Since $t0 = $zero, beq is taken to L1.

(b) The branch skips add and transfers control to L1, so sub is executed.

(c) $s2 = 4 − 9 = −5.

Key Point: slt produces 0 or 1; a following branch can use that result to control program flow.

Exercise 4 | If–Else with Comparison

Assume: a → $s0, b → $s1, x → $s2, y → $s3

if (a < b) x = x + y; else x = x − y;

(a) Use slt to evaluate a < b. What value represents true or false?

(b) Write the complete MIPS code using slt, a conditional branch, and j.

(c) If a = 7, b = 12, x = 20, and y = 5, determine the final value of x.

Answer

(a) slt $t0, $s0, $s1 → $t0 = 1 if a < b; otherwise $t0 = 0.

(b) slt $t0, $s0, $s1

```
beq $t0, $zero, Else ; add $s2, $s2, $s3
j Exit ; Else: sub $s2, $s2, $s3 ; Exit:
```

(c) 7 < 12, so $t0 = 1 and the branch is not taken. x = 20 + 5 = 25.

Key Point: Relational conditions such as < can be built from slt plus conditional branches.

Exercise 5 | Array Loop Analysis

Assume: i → $s0, k → $s1, base(A) → $s2
i = 1, k = 8, base(A) = 1000
A = {3, 8, 8, 5, 8}; each element is a 32-bit word.

```
Loop: sll  $t0, $s0, 2
      add  $t0, $t0, $s2
      lw   $t1, 0($t0)
      bne  $t1, $s1, Exit
      addi $s0, $s0, 1
      j    Loop
Exit:
```

(a) In the first iteration, what address is accessed and what value is loaded into $t1?

(b) Trace the loaded values and determine when bne is taken.

(c) How many times is addi executed, and what is the final value of i?

Exercise 5 | Array Loop Analysis — Answer

Answer

(a) i = 1 → offset = 1 × 4 = 4.
Address = 1000 + 4 = 1004; $t1 = A[1] = 8.

(b) i = 1: load 8 → bne not taken → i becomes 2.
i = 2: load 8 → bne not taken → i becomes 3.
i = 3: load 5 → 5 ≠ 8 → bne is taken to Exit.

(c) addi executes twice. The loop exits with i = 3.

Key Point: Array loops combine word-offset calculation, memory access, comparison, and branch control.